

Java 8 Stream API — Complete Notes

1. What is Stream API?

The **Stream API** was introduced in **Java 8** to process collections of data in a **functional and declarative way**.

A Stream is **not a data structure**. It does not store data. Instead, it provides a way to process data from sources such as:

- List
- Set
- Map
- Arrays
- Other data sources

Example

Traditional approach:

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);

for (Integer n : numbers) {
    if (n > 20) {
        System.out.println(n);
    }
}
```

Using Stream API:

```
numbers.stream()
    .filter(n -> n > 20)
    .forEach(System.out::println);
```

Output:

```
30
40
```

Main advantage

Stream API allows us to write:

What should happen?

instead of writing every step of:

How should it happen?

2. Why was Stream API introduced?

Before Java 8, collection processing usually required loops.

For example:

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);  
  
for (Integer n : numbers) {  
    if (n % 2 == 0) {  
        System.out.println(n);  
    }  
}
```

Stream API provides a cleaner approach:

```
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Benefits

- Less code
- Readable code
- Functional programming style
- Easy data transformation
- Easy filtering
- Easy aggregation
- Supports parallel processing

3. Collection vs Stream

This is a very common interview question.

Collection	Stream
Stores data	Processes data
Data structure	Processing mechanism
Can be reused	Generally cannot be reused
Eagerly stores elements	Lazily processes operations
Can add/remove elements	Does not modify source by itself

Example

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30);
```

The `List` stores the data.

```
numbers.stream()
```

creates a Stream for processing that data.

4. Stream Pipeline

A Stream generally follows this structure:

```
SOURCE
  ↓
stream()
  ↓
INTERMEDIATE OPERATIONS
  ↓
TERMINAL OPERATION
  ↓
RESULT
```

Example:

```
List<Integer> numbers =
    Arrays.asList(10, 15, 20, 25, 30);

List<Integer> result =
    numbers.stream()
        .filter(n -> n % 2 == 0)
```

```
.map(n -> n * 2)
.collect(Collectors.toList());
```

Pipeline:

```
List
↓
stream()
↓
filter()
↓
map()
↓
collect()
↓
List
```

5. Types of Stream Operations

There are mainly two types.

Intermediate Operations

Intermediate operations return another Stream.

Examples:

```
filter()
map()
sorted()
distinct()
limit()
skip()
```

They are generally **lazy**.

Terminal Operations

Terminal operations produce the final result or action.

Examples:

```
forEach()  
collect()  
count()  
min()  
max()  
reduce()  
findFirst()  
findAny()  
anyMatch()  
allMatch()  
noneMatch()  
toArray()
```

Once a terminal operation is executed, the Stream is consumed.

6. Creating a Stream

From Collection

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30);  
  
Stream<Integer> stream =  
    numbers.stream();
```

From Set

```
Set<String> names =  
    new HashSet<>();  
  
names.add("Amit");  
names.add("Rahul");  
  
names.stream()  
    .forEach(System.out::println);
```

From Array

```
int[] numbers = {10, 20, 30};
```

```
Arrays.stream(numbers)
    .forEach(System.out::println);
```

From Values

```
Stream<Integer> stream =
    Stream.of(10, 20, 30);
```

7. filter()

`filter()` is used to **select elements based on a condition**.

It takes a:

```
Predicate<T>
```

A Predicate returns:

```
boolean
```

Syntax

```
stream.filter(element -> condition)
```

Example

```
List<Integer> numbers =
    Arrays.asList(10, 15, 20, 25, 30);

numbers.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
10
20
30
```

Another example

```
numbers.stream()
    .filter(n -> n > 20)
    .forEach(System.out::println);
```

Output:

```
25
30
```

Multiple conditions

```
numbers.stream()
    .filter(n -> n > 10 && n < 30)
    .forEach(System.out::println);
```

Memory trick

```
filter() → SELECT
```

8. map()

`map()` is used to **transform each element**.

Syntax

```
stream.map(element -> transformation)
```

Example:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30);

numbers.stream()
    .map(n -> n * 2)
    .forEach(System.out::println);
```

Output:

```
20  
40  
60
```

String example

```
List<String> names =  
    Arrays.asList("java", "spring", "react");  
  
names.stream()  
    .map(name -> name.toUpperCase())  
    .forEach(System.out::println);
```

Output:

```
JAVA  
SPRING  
REACT
```

Method reference

```
.map(String::toUpperCase)
```

Memory trick

```
map() → TRANSFORM
```

9. filter() + map()

These two are frequently used together.

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);  
  
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .map(n -> n * 10)  
    .forEach(System.out::println);
```

Output:

```
100  
200  
300
```

Pipeline:

```
10 15 20 25 30  
  ↓  
  filter  
  ↓  
10 20 30  
  ↓  
  map  
  ↓  
100 200 300
```

10. `forEach()`

`forEach()` is a **terminal operation**.

It is used to perform an action on every element.

Syntax

```
stream.forEach(element -> action);
```

Example:

```
numbers.stream()  
    .forEach(n -> System.out.println(n));
```

Using method reference:

```
numbers.stream()  
    .forEach(System.out::println);
```

Important

`forEach()` returns:

```
void
```

Therefore:

```
numbers.stream()  
    .forEach(System.out::println);
```

cannot be followed by another Stream operation.

Memory trick

```
forEach() → USE / PROCESS
```

11. collect()

`collect()` is a terminal operation used to collect Stream results into a collection or another result.

Example:

```
List<Integer> result =  
    numbers.stream()  
        .filter(n -> n > 20)  
        .collect(Collectors.toList());
```

Output:

```
[25, 30]
```

Stream → List

```
List<Integer> result =  
    numbers.stream()  
        .collect(Collectors.toList());
```

Stream → Set

```
Set<Integer> result =  
    numbers.stream()  
        .collect(Collectors.toSet());
```

Memory trick

```
collect() → STORE
```

12. sorted()

`sorted()` is used to sort stream elements.

It is an intermediate operation.

Ascending order

```
List<Integer> numbers =  
    Arrays.asList(40, 10, 30, 20);  
  
numbers.stream()  
    .sorted()  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30  
40
```

Descending order

```
numbers.stream()  
    .sorted(Comparator.reverseOrder())  
    .forEach(System.out::println);
```

Output:

```
40  
30  
20  
10
```

13. Sorting Strings

```
List<String> names =  
    Arrays.asList("Rahul", "Amit", "Prashant");  
  
names.stream()  
    .sorted()  
    .forEach(System.out::println);
```

Output:

```
Amit  
Prashant  
Rahul
```

14. distinct()

`distinct()` removes duplicate elements.

Example:

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 10, 30, 20, 40);  
  
numbers.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30  
40
```

Memory trick

```
distinct() → REMOVE DUPLICATES
```

15. limit()

`limit(n)` returns only the first `n` elements.

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30, 40, 50);  
  
numbers.stream()  
    .limit(3)  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30
```

Memory trick

`limit()` → TAKE

16. skip()

`skip(n)` skips the first `n` elements.

```
numbers.stream()  
    .skip(2)  
    .forEach(System.out::println);
```

Output:

```
30  
40  
50
```

Memory trick

`skip()` → IGNORE

17. count()

`count()` returns the number of elements.

It returns:

`long`

Example:

```
long count =  
    numbers.stream()  
            .count();  
  
System.out.println(count);
```

Output:

5

With filter

```
long count =  
    numbers.stream()  
            .filter(n -> n > 20)  
            .count();
```

18. min()

`min()` finds the smallest element.

It returns:

`Optional<T>`

Example:

```
List<Integer> numbers =  
    Arrays.asList(30, 10, 50, 20);
```

```
Optional<Integer> min =
    numbers.stream()
        .min((a, b) -> a.compareTo(b));

System.out.println(min.orElse(0));
```

Output:

10

Method reference

```
.min(Integer::compareTo)
```

is equivalent to:

```
.min((a, b) -> a.compareTo(b))
```

19. `max()`

`max()` finds the largest element.

```
Optional<Integer> max =
    numbers.stream()
        .max((a, b) -> a.compareTo(b));

System.out.println(max.orElse(0));
```

Output:

50

20. Why does `min()` / `max()` return `Optional`?

The Stream could be empty.

Example:

```
List<Integer> numbers =  
    new ArrayList<>();
```

There is no minimum value.

Therefore:

```
Optional<Integer>
```

can represent:

```
Value exists    → Optional contains value  
No value       → Optional.empty()
```

21. Optional

`Optional<T>` is a container that may or may not contain a value.

Example:

```
Optional<Integer> value =  
    Optional.of(10);
```

Check whether value exists

```
value.isPresent()
```

Get value

```
value.get()
```

Safer approach

```
value.orElse(0)
```

Important

Avoid blindly using:

```
optional.get()
```

because it can throw `NoSuchElementException` when empty.

22. `findFirst()`

`findFirst()` returns the first element.

Return type:

```
Optional<T>
```

Example:

```
Optional<Integer> result =  
    numbers.stream()  
        .findFirst();  
  
System.out.println(result.orElse(0));
```

23. `findAny()`

`findAny()` returns any element from the stream.

```
Optional<Integer> result =  
    numbers.stream()  
        .findAny();  
  
System.out.println(result.orElse(0));
```

For sequential streams, it often appears to return the first element, but you should not rely on that behavior.

`findAny()` is particularly useful with parallel streams.

24. findFirst() vs findAny()

findFirst()	findAny()
Returns first element	Returns any element
Respects encounter order	Does not guarantee first
Useful when order matters	Useful when order doesn't matter

25. anyMatch()

Checks whether **at least one** element matches a condition.

Returns:

```
boolean
```

Example:

```
boolean result =  
    numbers.stream()  
        .anyMatch(n -> n > 40);  
  
System.out.println(result);
```

Output:

```
true
```

Because 50 is greater than 40.

Memory trick

```
anyMatch() → AT LEAST ONE
```

26. allMatch()

Checks whether **all elements** match the condition.

```
boolean result =  
    numbers.stream()  
        .allMatch(n -> n > 0);
```

If every number is positive:

true

Memory trick

allMatch() → EVERY ONE

27. noneMatch()

Checks whether **no element** matches the condition.

```
boolean result =  
    numbers.stream()  
        .noneMatch(n -> n < 0);
```

Output:

true

if there are no negative numbers.

Memory trick

noneMatch() → NO ONE

28. reduce()

`reduce()` combines multiple stream elements into a **single result**.

It is a terminal operation.

Example: Sum

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30, 40);  
  
Optional<Integer> sum =  
    numbers.stream()  
        .reduce((a, b) -> a + b);  
  
System.out.println(sum.orElse(0));
```

Output:

```
100
```

29. `reduce()` with Identity

You can provide an initial value.

```
int sum =  
    numbers.stream()  
        .reduce(0, (a, b) -> a + b);  
  
System.out.println(sum);
```

Output:

```
100
```

Here:

```
0
```

is the identity value.

Common identity values

```
Addition      → 0  
Multiplication → 1
```

30. Product using `reduce()`

```
List<Integer> numbers =  
    Arrays.asList(2, 3, 4);  
  
int product =  
    numbers.stream()  
        .reduce(1, (a, b) -> a * b);  
  
System.out.println(product);
```

Output:

24

31. Maximum using `reduce()`

```
Optional<Integer> max =  
    numbers.stream()  
        .reduce((a, b) ->  
            a > b ? a : b);
```

32. `mapToInt()`

`mapToInt()` converts a:

`Stream<Integer>`

into:

`IntStream`

Example:

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30);  
  
int sum =
```

```
numbers.stream()
    .mapToInt(n -> n)
    .sum();

System.out.println(sum);
```

Output:

```
60
```

33. Why use `mapToInt()`?

`IntStream` provides convenient numeric operations:

```
sum()
average()
min()
max()
count()
```

Example:

```
double average =
    numbers.stream()
        .mapToInt(n -> n)
        .average()
        .orElse(0);
```

34. `IntStream`

You can directly create an `IntStream`.

```
IntStream.range(1, 5)
    .forEach(System.out::println);
```

Output:

```
1
2
```

```
3  
4
```

Notice that 5 is excluded.

35. `rangeClosed()`

```
IntStream.rangeClosed(1, 5)  
          .forEach(System.out::println);
```

Output:

```
1  
2  
3  
4  
5
```

Difference

```
range(1,5)      → 1 2 3 4  
rangeClosed(1,5) → 1 2 3 4 5
```

36. Stream → Array

Object array

```
Object[] arr =  
    numbers.stream()  
            .toArray();
```

Integer array

```
Integer[] arr =  
    numbers.stream()  
            .toArray(Integer[]::new);
```

Primitive int array

```
int[] arr =
    numbers.stream()
        .mapToInt(n -> n)
        .toArray();
```

37. Stream → String

Use:

```
Collectors.joining()
```

Example:

```
List<String> names =
    Arrays.asList("Java", "Spring", "React");

String result =
    names.stream()
        .collect(Collectors.joining(", "));

System.out.println(result);
```

Output:

```
Java, Spring, React
```

With prefix and suffix

```
String result =
    names.stream()
        .collect(
            Collectors.joining(", ", "[", "]")
        );
```

Output:

```
[Java, Spring, React]
```

38. Important Collectors

The `Collectors` class provides many useful collectors.

Common ones:

```
toList()
toSet()
toMap()
joining()
groupingBy()
partitioningBy()
counting()
summarizingInt()
mapping()
```

39. `Collectors.toList()`

Converts Stream into a List.

```
List<Integer> result =
    numbers.stream()
        .filter(n -> n > 20)
        .collect(Collectors.toList());
```

40. `Collectors.toSet()`

Converts Stream into a Set and removes duplicates.

```
List<Integer> numbers =
    Arrays.asList(10, 20, 10, 30);

Set<Integer> result =
    numbers.stream()
        .collect(Collectors.toSet());
```

Result:

```
[10, 20, 30]
```

41. Collectors.joining()

Combines Strings.

```
String result =
    names.stream()
        .collect(Collectors.joining(", "));
```

42. groupingBy()

One of the **most important Stream API interview topics**.

`groupingBy()` groups elements based on a property.

Suppose we have:

```
class Employee {

    private String name;
    private String department;

    public Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }
}
```

Employees:

```
List<Employee> employees =
    Arrays.asList(
        new Employee("Amit", "IT"),
        new Employee("Rahul", "HR"),
        new Employee("Ravi", "IT"),
```

```
        new Employee("Priya", "HR")
    );
```

Group employees by department:

```
Map<String, List<Employee>> result =
    employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::getDepartment
            )
        );
```

Result conceptually:

```
IT → [Amit, Ravi]
HR → [Rahul, Priya]
```

Memory trick

```
groupingBy() → GROUP
```

43. `groupingBy()` with Counting

Suppose we want the number of employees in each department.

```
Map<String, Long> result =
    employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
            )
        );
```

Result:

```
IT → 2
HR → 2
```

44. partitioningBy()

`partitioningBy()` divides elements into exactly **two groups** based on a boolean condition.

Example:

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);  
  
Map<Boolean, List<Integer>> result =  
    numbers.stream()  
        .collect(  
            Collectors.partitioningBy(  
                n -> n % 2 == 0  
            )  
        );
```

Conceptually:

```
true  → [10, 20, 30]  
false → [15, 25]
```

Difference

```
groupingBy()    → multiple groups  
partitioningBy() → two groups: true / false
```

45. counting()

Counts elements.

Example:

```
long count =  
    names.stream()  
        .collect(Collectors.counting());
```

More commonly used with `groupingBy()`:

```
Map<String, Long> result =  
    employees.stream()  
        .collect(  
            Collectors.groupingBy(  
                Employee::getDepartment,  
                Collectors.counting()  
            )  
        );
```

```
        Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.counting()  
        )  
    );
```

46. summarizingInt()

Provides multiple statistics at once.

```
IntSummaryStatistics stats =  
    numbers.stream()  
        .collect(  
            Collectors.summarizingInt(  
                n -> n  
            )  
        );
```

You can get:

```
stats.getCount();  
stats.getSum();  
stats.getMin();  
stats.getMax();  
stats.getAverage();
```

47. mapping() Collector

`mapping()` can transform elements while collecting.

Example:

```
Map<String, List<String>> result =  
    employees.stream()  
        .collect(  
            Collectors.groupingBy(  
                Employee::getDepartment,  
                Collectors.mapping(  
                    Employee::getName,  
                    Collectors.toList()  
                )  
            )  
        );
```

```
)  
);
```

Result:

```
IT → [Amit, Ravi]  
HR → [Rahul, Priya]
```

48. Stream with Objects

Streams become very powerful with objects.

Example:

```
class Employee {  
  
    private String name;  
    private double salary;  
  
    // constructor, getters  
}
```

Find employees with salary greater than 50,000:

```
employees.stream()  
    .filter(e -> e.getSalary() > 50000)  
    .forEach(e -> System.out.println(e.getName()));
```

Get only employee names:

```
List<String> names =  
    employees.stream()  
        .map(Employee::getName)  
        .collect(Collectors.toList());
```

Find the highest salary:

```
Optional<Employee> employee =  
    employees.stream()  
        .max((e1, e2) ->  
            Double.compare(  
                e1.getSalary(),
```

```
e2.getSalary()  
));
```

49. Lazy Evaluation

Intermediate operations are **lazy**.

Example:

```
numbers.stream()  
    .filter(n -> {  
        System.out.println("Filtering " + n);  
        return n > 20;  
    });
```

Nothing happens yet because there is no terminal operation.

Once we add:

```
.forEach(System.out::println);
```

the pipeline executes.

Important

```
Intermediate operation  
    ↓  
    Lazy  
  
Terminal operation  
    ↓  
    Starts execution
```

50. Stream Cannot Normally Be Reused

Once a terminal operation is executed, the Stream is consumed.

Example:

```
Stream<Integer> stream =  
    numbers.stream();
```

```
stream.count();

stream.forEach(System.out::println);
```

This causes:

```
IllegalStateException
```

Correct approach

Create a new Stream:

```
numbers.stream().count();

numbers.stream().forEach(System.out::println);
```

51. Stream Does Not Modify Original Collection

Example:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30);

List<Integer> result =
    numbers.stream()
        .map(n -> n * 2)
        .collect(Collectors.toList());
```

Original:

```
[10, 20, 30]
```

Result:

```
[20, 40, 60]
```

The original List is not changed by the Stream pipeline.

52. Short-Circuit Operations

Some Stream operations can stop processing early.

Examples:

```
limit()
findFirst()
findAny()
anyMatch()
allMatch()
noneMatch()
```

Example:

```
boolean result =
    numbers.stream()
        .anyMatch(n -> n > 20);
```

Once a matching element is found, processing can stop.

53. Stream Example — Complete Pipeline

Find the first three unique even numbers greater than 10, sort them, and store them in a List.

```
List<Integer> result =
    numbers.stream()
        .filter(n -> n > 10)
        .filter(n -> n % 2 == 0)
        .distinct()
        .sorted()
        .limit(3)
        .collect(Collectors.toList());
```

Pipeline:

```
Source
↓
filter()
↓
filter()
↓
distinct()
↓
```

```
sorted()
↓
limit()
↓
collect()
↓
List
```

54. Parallel Stream

A Stream can be processed in parallel.

```
numbers.parallelStream()
    .forEach(System.out::println);
```

Or:

```
numbers.stream()
    .parallel()
    .forEach(System.out::println);
```

Important

Parallel streams can improve performance for suitable large computational workloads, but they are **not automatically faster**.

Also, output order with `forEach()` may not be guaranteed.

If encounter order matters:

```
numbers.parallelStream()
    .forEachOrdered(System.out::println);
```

55. Sequential vs Parallel Stream

Sequential	Parallel
One processing path	Multiple processing paths
<code>stream()</code>	<code>parallelStream()</code>
Simpler behavior	More overhead

Sequential

Parallel

Usually suitable for ordinary collections

Useful for suitable large workloads

56. `forEach()` vs `forEachOrdered()`

```
stream.forEach(...)
```

does not guarantee encounter order for a parallel stream.

```
stream.forEachOrdered(...)
```

preserves encounter order.

57. `map()` vs `filter()`

Very common interview question.

<code>filter()</code>	<code>map()</code>
Selects elements	Transforms elements
Uses Predicate	Uses Function
Returns same element type generally	Can change element type
Condition-based	Transformation-based

Example:

```
.filter(n -> n > 10)
```

means:

```
Select numbers > 10
```

Whereas:

```
.map(n -> n * 2)
```

means:

Transform every number

58. `map()` vs `mapToInt()`

`map()`

Returns another object Stream.

```
Stream<Integer> result =  
    numbers.stream()  
        .map(n -> n * 2);
```

`mapToInt()`

Returns an `IntStream`.

```
IntStream result =  
    numbers.stream()  
        .mapToInt(n -> n);
```

`IntStream` provides:

```
sum()  
average()  
min()  
max()
```

59. `reduce()` vs `collect()`

`reduce()`

Used to combine elements into one result.

```
int sum =  
    numbers.stream()  
        .reduce(0, (a, b) -> a + b);
```

Result:

100

`collect()`

Used to gather elements into a collection/result.

```
List<Integer> result =  
    numbers.stream()  
        .collect(Collectors.toList());
```

Memory trick

`reduce()` → ONE RESULT
`collect()` → COLLECTION / RESULT CONTAINER

60. `findFirst()` vs `min()`

These are not the same.

`findFirst()`

returns the first element based on encounter order.

`min()`

returns the smallest element according to the comparator.

Example:

[50, 10, 30]

```
findFirst() → 50  
min()      → 10
```

61. `anyMatch()` vs `filter()`

`filter()` returns matching elements as a Stream.

```
List<Integer> result =  
    numbers.stream()  
        .filter(n -> n > 20)  
        .collect(Collectors.toList());
```

`anyMatch()` only tells whether at least one element matches.

```
boolean result =  
    numbers.stream()  
        .anyMatch(n -> n > 20);
```

62. Common Mistakes

Mistake 1: Forgetting terminal operation

```
numbers.stream()  
    .filter(n -> n > 10);
```

Nothing is actually consumed from the pipeline.

Correct:

```
numbers.stream()  
    .filter(n -> n > 10)  
    .forEach(System.out::println);
```

Mistake 2: Reusing a Stream

```
Stream<Integer> stream =  
    numbers.stream();  
  
stream.count();  
  
stream.forEach(System.out::println);
```

Wrong because the Stream has already been consumed.

Mistake 3: Confusing `filter()` and `map()`

Wrong concept:

```
.map(n -> n > 10)
```

if your intention is to remove elements.

Use:

```
.filter(n -> n > 10)
```

Mistake 4: Using `get()` blindly

```
min.get();
```

can throw an exception when Optional is empty.

Prefer:

```
min.orElse(0);
```

when an appropriate default exists.

Mistake 5: Assuming parallel streams are always faster

They are not.

For small collections, parallel processing can add unnecessary overhead.

63. Most Important Stream Operations Cheat Sheet

```
filter()      → SELECT
map()         → TRANSFORM
sorted()      → SORT
distinct()    → REMOVE DUPLICATES
limit()       → TAKE FIRST N
```

<code>skip()</code>	→ SKIP FIRST N
<code>forEach()</code>	→ PERFORM ACTION
<code>collect()</code>	→ STORE RESULT
<code>count()</code>	→ COUNT
<code>min()</code>	→ SMALLEST
<code>max()</code>	→ LARGEST
<code>findFirst()</code>	→ FIRST
<code>findAny()</code>	→ ANY
<code>reduce()</code>	→ COMBINE
<code>anyMatch()</code>	→ AT LEAST ONE
<code>allMatch()</code>	→ ALL
<code>noneMatch()</code>	→ NONE
<code>toArray()</code>	→ ARRAY
<code>joining()</code>	→ STRING
<code>groupingBy()</code>	→ GROUP
<code>partitioningBy()</code>	→ TWO GROUPS

64. Important Functional Interfaces Used with Streams

Operation	Functional Interface	Purpose
<code>filter()</code>	<code>Predicate<T></code>	Returns boolean
<code>map()</code>	<code>Function<T,R></code>	Converts $T \rightarrow R$
<code>forEach()</code>	<code>Consumer<T></code>	Performs action
<code>sorted()</code>	<code>Comparator<T></code>	Compares elements
<code>reduce()</code>	<code>BinaryOperator<T></code>	Combines two values

Examples

```
.filter(n -> n > 10)
```

Predicate:

```
T → boolean
```

```
.map(n -> n * 2)
```

Function:

```
T → R
```

```
.forEach(n -> System.out.println(n))
```

Consumer:

```
T → void
```

65. Method References in Streams

Method references make Stream code shorter.

Lambda

```
.map(name -> name.toUpperCase())
```

Method reference:

```
.map(String::toUpperCase)
```

Lambda

```
.forEach(n -> System.out.println(n))
```

Method reference:

```
.forEach(System.out::println)
```

Lambda

```
.map(e -> e.getName())
```

Method reference:

```
.map(Employee::getName)
```

Lambda

```
.min((a, b) -> a.compareTo(b))
```

Method reference:

```
.min(Integer::compareTo)
```

66. Real-World Example

Suppose we have employees:

```
List<Employee> employees = ...;
```

Find names of employees whose salary is greater than ₹50,000:

```
List<String> result =  
    employees.stream()  
        .filter(e -> e.getSalary() > 50000)  
        .map(Employee::getName)  
        .collect(Collectors.toList());
```

This is a very common interview-style Stream problem.

Pipeline:

```
Employees  
  ↓  
filter salary > 50000  
  ↓  
Employee objects  
  ↓  
map to names  
  ↓  
List<String>
```

67. Another Real-World Example

Find the highest-paid employee:

```
Optional<Employee> highest =  
    employees.stream()  
        .max((e1, e2) ->  
            Double.compare(  
                e1.getSalary(),  
                e2.getSalary()  
            ));
```

Get the employee safely:

```
highest.ifPresent(  
    e -> System.out.println(e.getName())  
);
```

68. Important Interview Questions

Beginner Level

1. What is Stream API?

Stream API is a Java 8 feature used to process collections and other data sources in a functional and declarative style.

2. Is Stream a collection?

No. A Stream does not store data; it processes data.

3. When was Stream API introduced?

Java 8.

4. What are intermediate operations?

Operations that return another Stream.

Examples:

```
filter()  
map()  
sorted()  
distinct()  
limit()  
skip()
```

5. What are terminal operations?

Operations that produce the final result or action.

Examples:

```
forEach()  
collect()  
count()  
reduce()  
min()  
max()
```

Intermediate Level

6. What is lazy evaluation?

Intermediate Stream operations are generally not executed until a terminal operation is invoked.

7. Can we reuse a Stream?

No. Once consumed by a terminal operation, the Stream cannot normally be reused.

8. Does Stream modify the original collection?

Stream operations do not inherently modify the source collection.

9. Difference between `filter()` and `map()` ?

`filter()` selects elements; `map()` transforms elements.

10. Difference between `findFirst()` and `findAny()` ?

`findFirst()` respects encounter order; `findAny()` can return any element.

Advanced Interview Questions

11. What is the difference between `map()` and `mapToInt()` ?

`map()` returns an object Stream, while `mapToInt()` returns an `IntStream`.

12. Why does `min()` return Optional?

Because the Stream can be empty.

13. What is `reduce()` ?

It combines Stream elements into a single result.

14. What is `groupingBy()` ?

It groups elements based on a classification function.

15. What is `partitioningBy()` ?

It divides elements into two groups based on a boolean condition.

16. What is a parallel Stream?

A Stream that can process elements using multiple processing paths.

17. Is a parallel Stream always faster?

No. It depends on the workload, data size, operation, and overhead.

69. Important Coding Questions for Practice

Basic

Q1. Find even numbers

```
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Q2. Find numbers greater than 50

```
numbers.stream()  
    .filter(n -> n > 50)  
    .forEach(System.out::println);
```

Q3. Convert numbers to double

```
numbers.stream()  
    .map(n -> n * 2)  
    .forEach(System.out::println);
```

Q4. Convert names to uppercase

```
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Q5. Remove duplicates

```
numbers.stream()  
    .distinct()  
    .forEach(System.out::println);
```

70. Intermediate Coding Questions

Practice these without looking at the solution:

1. Find all odd numbers.
2. Find numbers greater than 25.
3. Find numbers between 20 and 50.
4. Find the smallest number.
5. Find the largest number.
6. Count even numbers.
7. Find the first number.
8. Find whether any number is greater than 100.
9. Check whether all numbers are positive.
10. Remove duplicate numbers.
11. Sort numbers in ascending order.
12. Sort numbers in descending order.
13. Find the first 5 numbers.
14. Skip the first 3 numbers.
15. Find the sum of all numbers.
16. Find the average.
17. Find the sum of even numbers.
18. Convert List to array.
19. Convert List of Strings into one String.
20. Find names starting with "A".

71. Advanced Coding Questions

Practice these for interviews:

1. Find the second-highest number.
2. Find the second-lowest number.
3. Find duplicate elements.

4. Find unique elements.
 5. Count frequency of each element.
 6. Find the most frequently occurring element.
 7. Find employees with salary greater than 50,000.
 8. Find the highest-paid employee.
 9. Find the lowest-paid employee.
 10. Group employees by department.
 11. Count employees in each department.
 12. Find the highest salary in each department.
 13. Find employees whose name starts with "A".
 14. Sort employees by salary.
 15. Sort employees by name.
 16. Find average employee salary.
 17. Partition employees based on salary.
 18. Convert employees to a Map.
 19. Find duplicate characters in a String.
 20. Count character frequency using Streams.
-

72. Complete Stream Example

```
import java.util.*;
import java.util.stream.*;

public class Main {

    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(
                10, 20, 10, 30, 40, 20, 50
            );

        List<Integer> result =
            numbers.stream()
                .filter(n -> n > 15)
                .distinct()
                .sorted()
                .map(n -> n * 2)
                .limit(3)
                .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Execution

Original:

10, 20, 10, 30, 40, 20, 50

After `filter()`:

20, 30, 40, 20, 50

After `distinct()`:

20, 30, 40, 50

After `sorted()`:

20, 30, 40, 50

After `map()`:

40, 60, 80, 100

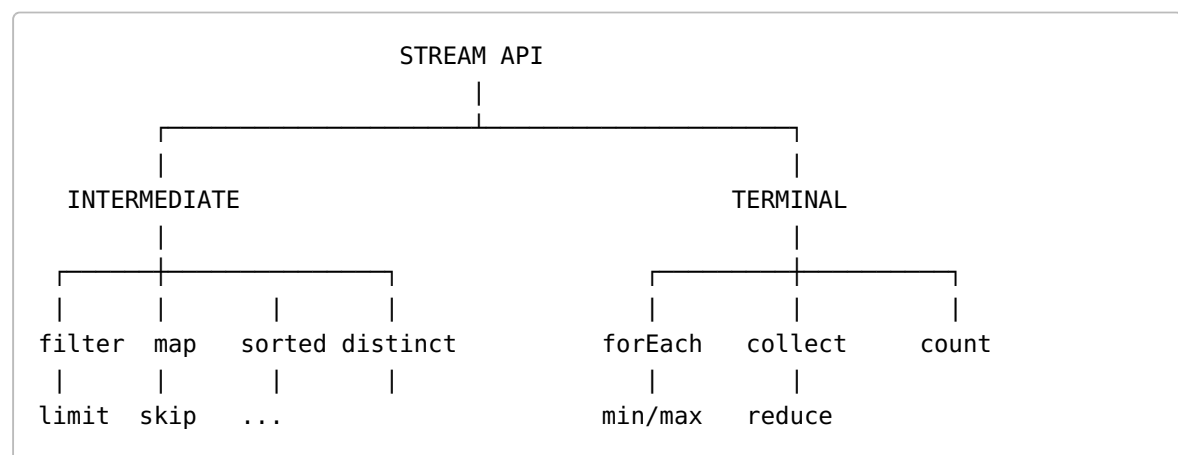
After `limit(3)`:

40, 60, 80

Final List:

[40, 60, 80]

73. Final Stream API Mind Map



|
findFirst/findAny
|
anyMatch/allMatch/noneMatch

74. Ultimate Stream API Cheat Sheet

SOURCE

↓

stream()

↓

INTERMEDIATE OPERATIONS	
filter()	→ Select
map()	→ Transform
sorted()	→ Sort
distinct()	→ Remove duplicates
limit()	→ First N
skip()	→ Skip N

↓

TERMINAL OPERATIONS	
forEach()	→ Perform action
collect()	→ Collect result
count()	→ Count
min()	→ Minimum
max()	→ Maximum
reduce()	→ Combine
findFirst()	→ First element
findAny()	→ Any element
anyMatch()	→ At least one
allMatch()	→ Every element
noneMatch()	→ No element
toArray()	→ Array

★ The 10 operations you MUST know for interviews

1. filter()
2. map()
3. collect()
4. sorted()
5. distinct()

```
6. min() / max()
7. reduce()
8. groupingBy()
9. anyMatch() / allMatch() / noneMatch()
10. mapToInt()
```

One-line memory

```
filter    → SELECT
map       → CHANGE
sorted    → ORDER
distinct  → UNIQUE
limit     → TAKE
skip      → IGNORE
collect   → STORE
forEach   → USE
min       → SMALLEST
max       → LARGEST
count     → COUNT
reduce    → ONE RESULT
grouping  → GROUP
joining   → STRING
```

Final Interview Rule

Whenever you see a Stream problem, think in this order:

```
1. What is my SOURCE?
   ↓
2. Do I need to FILTER?
   ↓
3. Do I need to TRANSFORM? → map()
   ↓
4. Do I need to SORT/REMOVE DUPLICATES?
   ↓
5. Do I need LIMIT/SKIP?
   ↓
6. What FINAL RESULT do I need?
   ↓
   collect / count / min / max /
   reduce / find / match / forEach
```

This approach will help you solve most beginner-to-intermediate Stream API interview problems.